lecture-week2-rec_compressed.mp4 transcript with timestamps

00:00
Okay, this is lecture week two for SIT112, Introduction to Data Science and AI. In this lecture we're going to talk about essentials for data analysis. My name is Davud, I'm a senior lecturer in software engineering at the School of Information Technology at Deakin University. So in the previous lecture we talked about data science and AI as interdisciplinary fields and the big overlap between these two fields. Analyzing data is part of that overlap and the focus of our analysis in this unit is textual data. So we analyze text. We consider textual data in a tabular format where we have rows and columns where each row represents that data entity. So the focus of this lecture would be on learning how to work with data frames, how to create data frames. Data frames are objects that will help us work with the data. They will model the data and they will work with this model instead of directly working with data. There are objects that represent the data or model the data and we work with the model in the code in the Python code. So we learned how to create a data frame either through directly reading the data from a file or by using the data frame constructor. And then how to save a data frame, how to examine the data, which means how to display the data and its attributes or how to use different methods such as info and unique and describe in order to examine the data that is in the data frame. We also learned how to access column source or a subset of columns and rows using different functions or methods, such as query or LOC or lock, ILock accesses. So there are methods and there are accesses that we can use. The methods include a query function and the accesses include an ILOC. Also we learn how to use Pandas methods to get statistics for the columns of a data frame. Continuing on the objectives of this lecture, we're going to learn how to perform calculations on data in the columns of a data frame, use replace method to replace the data in a data frame, use pandas methods to do the following operations on the data frame. That includes sort, set and reset an index, pivoting, melting, grouping and plotting some basic visualizations from the data.

03:14
Alright let's begin with an introduction to Pandas DataFrame. So we talked about Pandas module in the previous lecture. We talked about different modules that come as part of the standard installation of Anaconda. One of the modules that will be included in your Python environment is pandas that you can use by default. And of cours

e you can always install modules when you need them. DataFrame is an object that c omes as part of Pandas module. So what is a DataFrame? A DataFrame or a DataFra me object is a Pandas object, so it's part of Pandas. That stores the data for analysis. So it models the data, it captures your data, and then you don't have to directly walk with the data for analysis. So it models the data, it captures your data, and then you don't have to directly walk with the data. You don't have to walk, for example, directl y with a CSV file. Instead, you'll be walking with a data frame. This object also provid es the attributes and methods for working with the data. So it's very powerful, very u seful object that you can not only model the data, but also manipulate the data, wor k with the data, and we'll see what are the functionalities that we can use from this o bject. Here you can see an example of what is stored in a data frame. Here you can s ee some rows starting from index zero going up to index four as represented here a nd more rows represented here. You can see five rows here and you can see each ro w has some attributes such as year, age group, and death rate. such as year, age gro up, and death rate. These are the attributes of the data entries or the elements store d in the data frame. So each row here represents a data entry or an element in the d ata frame or a data entity or an instance of the data entity. entity and or an instance of the data entity and each column specifies an attribute of the data frame. The data frame components, so these are the main components of a data frame. Column label s, column data, column data types, index and metadata. So column labels are the na mes or the labels of the columns or the names of the attributes of the data that are s tored in a data frame. There's column data, that is the data in the columns. All of the data in a column typically has the same data type with one entry in each row. And co lumn data types specify the types of the data that are stored in the column, for exam ple string or date time, different types that exist. The index also known as a row label , if an index is not defined it is generated as a sequence of integers starting from zer o, as we saw in the previous example. And the metadata. So metadata are attributes of the data frame that are generated by pandas when the data frame is constructed or changed.

06:52
Alright, how to read data into a data frame. So this is how we can create a data fram e by reading the data from a file. That file of course can come in different formats. If it's a CSV file we're gonna use read CSV function which is going to read the contents of the CSV file and create a data frame form from that content. Alternatively, we can

use read Excel if your data is in an Excel format, read SQL, JSON, HTML, and pickle if the format of your data is any of those. I will talk about some of these formats in this lecture and we'll see some examples in the future lectures. Here you can see some ex amples of reading the contents of different files into a data frame. So PD, PD if you r emember from the previous lecture is a reference to Pandas module. So when you i mport your Pandas module, like import Pandas, then after that you have as some na me and that name here is PD could be anything that you refer that you use to refer t o pandas module so PD.readcsv is going to call the read CSV function or method for m pandas and it's going to pass this parameter mydata.csv, which is the name of the file or the full address to the file. When we pass the name of the file, we assume that the file is in the current directory, the current folder. Otherwise we need to pass the f ull name of the file including the full path. And then, so this is going to read the cons t of the CSV file into a data frame and that data frame is named DF. See another exa mple here where we read the const of an Excel file into the same data frame df that's a new data frame but it's named the same df of course it's going to if you execute th ese lines consecutively df here no longer contains the initial contents the contents wi ll be replaced by the contents of the new data frame that is created by the read excel file. So every time you call read csv or read excel or read sql they're going to create a new data frame and return data frame and you're going to reserve to that data fram e as df. So here's another example where there's a connection object being created h ere, create engine blah. So this creates a connection to this specific database and the n the connection object is passed to the read sql function. This is going to use the co nnection to extract all the rows from this particular table and then create a data fram e which would be referred to as DF. Another example here is a reading gate or creati ng a data frame from a JSON file, from an HTML file and from a Pico file. So you see the mechanism is similar. You can read the contents of a file that's available online, le t's say a CSV file for instance, into a data frame by providing the direct link to the file and of course that direct link is a URL. So here you have a URL that refers to a specifi c CSV file and then we're passing that URL as the input of the read CSV function. Wh at the function does is that it accesses the CSV file through the URL, reads the conte nts of the CSV file and then creates a data frame to model the data and returns the d ata frame which will be referred to as mortality underlined data of course this is a na me and we can set any name that we like to refer to the data frame that is returned by the read CSV function and eventually we have the name of the data frame listed h ere mortality underlying data what this line does in a cell in your Jupyter notebook t

his is going to display the contents of the data frame and of course if you're having hundreds of rows or thousands of rows it's not going to display all of them it's going to display the first, usually five rows, and the last five rows. Of course, you can chang e that by setting the parameters to customize it. Another way to display the contents of the data frame is to use the display function and pass the data frame as the input of the display function. And if you just want to see what kind of data is stored there, you can use the head function, which we'll see examples of that, which is going to ju st display the first five rows of the data frame.

12:24
There's another way to create a data frame and that is by using its constructor. Befor e talking about that, what is a constructor? Constructor comes from the concept of o bject-oriented programming. In object-oriented programming, every object represen ts an entity in real world. And when we want to instantiate that object, create an inst ance of the object, we call the constructor of the object. And the constructor is going to initiate an instance of the object and load it into the memory so we can work with it. So that's what a constructor does. In the case of a data frame, a data frame is an o bject and every time we want to create a new data frame or we want to instantiate a data frame, we can use the constructor the data frame. This is one way. There's anot her way and that is to read the console.file directly and for example using the read C SV function and that will create a data frame for us. So if you're looking at an alterna tive way to do that and that is to directly calling the constructor and of course if we do so we need to pass or prepare the values for the parameters of the function. That includes the data, the columns, and index. What is data? Well, obviously, it's the data that DataFrame is going to model. It's to specify the data that will be used to create a DataFrame. It can be a list, dictionary, another data frame or a non-py array. The da ta can be modeled in different ways using different data structure. And there's anoth er one, the next parameter is columns. That is to specify the column labels for the da ta frame and it can be a list or an array. And the last parameter here is an index. It ha s to specify the row labels for the data frame. It can be a list or an array. Here we can see an example of creating a data frame using its constructor. So here we have a df u nderline data equals a list of lists. You can see there is a list that contains two elemen ts and each element itself is a list of values. So you have a list of lists. In a way it's like a matrix. In other words, it's a table. So it has two rows and three columns. And of co urse the first row will be index 0, the second row will be index 1, the first column wou

ld be index 0, second column would be index 1, and the third column would be inde x 2. And of course each column has its own value at the intersection of each row and each column. So this is our data captured as a list of lists in Python. And then we hav e a list here that represents the column names. And then we can call the constructor of the data frame by directly accessing the constructor form PD. What is PD? PD is o ur reference to Pandas module. Remember, import Pandas as something and we ass ume that is imported as PD. It could be any name that we assign. So now we are refe rring to the constructor of data frame using this PD reference to the Pandas module. So PD.dataFrame is going to call the constructor of data frame and then we pass the parameters of the data frame or the parameters of the data frame that we want to cr eate, we want to instantiate, which includes the data and the column names. So we c reate a data frame this way using the constructor. The constructor returns a data fra me. We name the data frame mortality underline df. The last step is to display the da ta frame. So see what's in there, mortality, underline, DF. Since there are only two ro ws, we only get two rows here.

16:56
Once we created the data frame, it's time to save the data frame. Why do we need to do that? It's because we want to be able to reuse it. Otherwise, once the program ter minates, the data frame will be gone because it's stored in the memory not in your p ersistent memory, not in your SSD or hard disk. So we need to store it so we can late r reuse it. Of course we can use different functions to do that such as toCSP, toExcel, toPico, toJSON, toHTML, toSQL Depending on the format that we want to store the data frame as. Here's an example. So df.toCSV is going to store your data frame, the contents of your data frame as a CSV file and this is the name of the CSV file, you pa ss the name of the CSV file. If you compare this with the read CSV file, here you have one more parameter index equals false. What does it do? So basically it says I don't want you to re-index my data frame, do not add a new column to my data frame to r e-index it. Otherwise if you don't set this parameter to false you're gonna get a new column which you probably don't need and then you need to get rid of that. So set t his to false will prevent from adding that index column. Similarly you can use other f unctions, so I'm not gonna pause here straightforward. So what are pql files? pql file s allow for serializable objects to be stored into a File and then they can be restored, they can be retrieved from the Pico file into objects. So it's in a way that could be, yo u could sort of hibernate them. It could go to sleep on the local drive and then when

you run, when you need them, you can bring them back to memory. You can make t hem live again as objects and work with them. So the Pico modules in Python provid es a way to serialize and deserialize. So serialize means writing objects as PQL files in to local drive, into your storage. And deserialization means bringing them back to lif e from local drive to a live object and it's a memory and so you can deserialize objec ts to and from a you can serialize and deserialize objects to and from a file. So seriali ze again is to store a serializable object into a pickle file and deserialization means re trieving that object from the pickle file and bringing it back to the memory as an obj ect. Pickling is the process of converting a Python object into a byte stream and unpi ckling is the inverse operation of loading the byte stream back into the Python objec t. It's another fancy way to say what I just explained. So no confusion. it's another fan cy way to say what I just explained. So no confusion. So the Pico module is used for v arious purposes such as saving and loading machine learning models. Once you crea te a machine learning model, you can store it on your storage, your hard disk, for ex ample or SSD and then you can restore the model and use it for example task of clas sification when you need it. Also for saving and loading large data structures and for caching intermediate results to disk. So here's an example of saving the data frame i nto a Pico file. First step is to save the data frame. So mortality underlined data is the data frame.toPico is going to store it as a Pico file. And where to? To this file named mortality underlined data.pico. This is the full path to the file. Since we have only pas sed the name of the file, this is going to store, create and store the file into the local directory, into the current, sorry, current directory or current path that the Jupyter no tebook file is being executed from. Otherwise you need to specify the full path. So th is stores the file there and of course if we want to retrieve the object or the data fra me in this case from the pickle file we need to use the read pickle function and then this last part displays the first five rows of the data frame.

22:09

So let's assume we read the contents of a CSV file into a data frame. Now it's time to look into the data frame and see what are the rows, what are the columns, what type of data is stored in the data frame. And maybe from there we may decide to get rid of some of the columns that we don't need or do some data cleaning, take care of s ome missing data and things like that. So we need to examine the data that is stored in the data frame so we can get some initial insights to see what we need to do with the data frame to clean it and prepare it for our data analysis. Let's see how we can d

o that. So first thing first, you may want to have an overview of the data and see wha t is in the data frame. Just have a look at the rows and columns. To be able to do that , you can just simply write the name of the data frame as your last statement in a cell in Jupyter Notebook and execute the cell. It will give you the first five rows and the la st five rows in the data frame. Of course you can customize that. And you can also se e that it has provided the number of the rows, 476 rows and three columns. You can also use the head function. So head function is going to display the first five rows of your data frame, which you can see here. Another function that you can use is a tail f unction. You can use a tail function to get a view of the last few rows of the data fra me. In this case we have passed value 3 as the input of the tail function which means I just want to see the last three rows of the data frame. If you want to see 4 of course you can set it 4. If you want to see less you can reduce the number so you can you c an customize the tail function. Alright so we can use the head and tail function to dis play the first X number of rows from the beginning of the data frame or a Y number of rows from the end of the data frame. We can also use, we can also customize the display function as given in this example, to display some number of rows from the b eginning and end of the data frame. In this case, we're using, we are limiting this cha nge to a limited scope. So we're doing this temporarily by using with PD dot option context that's the function we're calling option context we're setting this display max rows to five and display max columns to none and then within this scope we're callin g the display function for this particular data frame mortality data. So what does it d o is that we are setting the maximum number of rows that are going to be displayed to five and we're setting the maximum number of columns to none which means ess entially we are saying do not put any restriction on the number of the columns, in ot her words display all the columns. So all the columns will be displayed and maximum of five rows will be displayed. So we set the maximum to five, what happens is that t he way the function display works is that it displays equal number of rows from the b eginning and from the end of the data frame. So when you set the limitation to 5, ob viously 5 cannot be divided by 2 so that we have equal number of rows at the begin ning of the end. But we can have two rows from the beginning and two rows from th e end and just ignore one row here. So we'll have, that's why the function has return ed four rows, like two rows from the beginning, two rows from the end. So even tho ugh displayMaxRows is set to five, we're actually getting four rows returned. If it cha nges to six, we'll get three rows from the beginning, three rows from the end. If we c hange to four, we get to 6 we'll get 3 rows from the beginning, 3 rows from the end.

If we change to 4 we get 2 rows from the beginning, 2 rows from the end. So in fact i t doesn't matter if we set this to 4 or 5 we get the same results displayed. And of cou rse you can see there are again 476 rows in the whole data frame but we're only vie wing 2 from the beginning and two from the end. As I said, the scope of this change is limited to score this function, option context column here. So this specifies the sco pe. We are, this indentation specifies the scope. We're calling display function within the scope of this change, and therefore, this has affected the way that the data fram e has been displayed here. So this change is temporary.

27:18
So the attributes of the data frame object. So a data frame object obviously has valu es, the values of the dataFrame in an array format, index, that's the row index, colum ns, size, that's the total number of elements in a dataFrame, and the shape, the num ber of rows and columns. Okay, now let's try and display the attributes of the dataFra me. We can start from values. Mortality data is a data frame. .values is going to displ ay the values of the data frame and these are the values. The values are displayed as an array here. So you have like a metrics format. You have rows and you have colum ns, so these are your rows and these are the columns. This is the way data is displaye d, which is not the best way you can display the data. And you can also try to print th e index, the columns, the size and the shape. So the index as you can see here starts from, goes all the way up to 476 with the step of one, which means you have 477 ele ments in that data frame. The columns are listed there, the labels of the columns are listed there, and the size, there are 1, 1428 values in the data frame and of course th e shape that is 476 rows by three columns. So the value here 1428 is calculated by m ultiplying these two values. So we have these number of rows and these number of c olumns which means you have the multiplication of these two numbers as the numb er of the values that are stored in your data frame. Okay, so here we can use the colu mns attribute to replace the spaces with nothing. Let's see what we mean by that. So here we have the data frame mortality underlined data. So we are accessing the colu mns attribute of the data frame. So mortality underlined data.columns equals, so we are setting the columns attribute of the data frame. So mortality underlined data dot columns equals. So we are setting the columns to what? To what comes now. Mortali ty dot underlined data is the same data frame dot columns. So same columns. But n ow we are changing the columns or the column labels by accessing the str attribute of the columns and then the function replace. So function replace is a function of str

which is an attribute of columns. columns.str.replace. So what's the replacement? Wh at are we replacing? We're replacing What are we replacing? We're replacing the em pty spaces in the column names, in the column labels, with nothing. In other words, we are removing the empty spaces in the column names. So we're connecting the w ords in the column names. Perhaps to make it easier to work with the data. perhaps t o make it easier to work with the data. So here we have displayed the columns of the data frame so we can see how the column names have changed. So this is the chang ed column names. The original column names, as you can see here, sorry, these are t he original ones, year, age, space group, death, space, rate, but now they have chang ed to year, age, group without any space, so the space has been removed, and then death rate, again the space has been removed, and you can see of course the update d column names here, year, age, group, death rate. So this is, so your data frame fro m this will change to this. We have only changed the column names, okay? We've on ly changed the column names. By replacing the empty spaces with nothing, we have connected two words in the column names. Of course, year hasn't changed because year is just one word, there's no empty space there and we only have here.

31:56
Okay three interesting functions info and unique and describe we can use this functi on to get more functions or methods to get more insights on our data or data frame . But what do they do? The info function returns information about the data frame a nd its columns. The nUnique function returns the number of unique data items in ea ch column, or in other words, returns the number of unique values in each column. A nd the describe function returns statistical information for each numeric column. So t he info method or info function returns some general information about the data fra me, about the structure, and how many entries are in the data frame, the number of non-null entities. So if the number of non-null entities is less than the total number o f entries, in other words, if one of these numbers was less than 476, it would imply th at there are some null values in the columns. You can see the columns of the data fra me are listed here, year, age group and death rate. And you can see that it says that the info function or info method tells us how many entries are in the data frame and their corresponding indices, which starts from zero and goes all the way up to 475. A nd it also tells us the types of the values in the columns. The first column here is an i nteger, and the second one, age group, is an object or a string, and the third one, de ath rate, is a float, which is another number with decimal points, another type of nu

mber with decimal points. We can also see the memory usage with different options, but these are the main things that we can get from the info function. It actually gives us the structure of the data frame, the types of the columns, and we can also quickly see if there are any null values. So, you know, if you need to deal with them and duri ng the data cleaning we might have to perhaps remove the rules with null values or substitute null values with some estimations or whatever technique we use. But this gives us a quick overview of the data frame, it's a very useful function. Using the nUn ique method or function, nUnique function is very useful, you can use it to see how many unique values exist in each column. Sometimes you want to process the values or you want to perform certain operations based on the values that are in the colum n, but you don't need all the reputations of those values, you just need to know how many unique values are there, or you want the exact unique values. So in this case yo u can use nUnique method, this of course only returns the number of unique items, and if you want to get the unique values you can call f another function which is a un ique, so just remove n, so dot unique function that is going to return a list of unique values within the column. So you can use n unique, n unique function in combinatio n together to get the number of unique values and the actual unique values if you n eed to process them or work with them. In this case we have called the end unique f unction. We've got 119 different years, unique year values, 4 unique age groups, 430 unique death rates and of course these are the three columns that we have in the da ta frame. Okay, so the almighty describe function. This is a very useful function which could quickly give you a description, a statistical values in your data frame and you c an use it as simple as using the name of the data frame followed by dot describe. So easy. And you get a beautiful table like this where you have your, the parameters, sta tistical parameters that you can get to describe your data frame and their correspon ding values for the numerical columns. In this case, you have two numerical columns here in death rate, and then you're getting the count, mean standard deviation, mea n, not M-E-A-N N but M I N minimum the first percentile 25 percent second percenti le 50 percent 75 the third percentile 75 percent and the maximum of values for each column so So let's see what they mean. So the first one, count, it tells us how many v alues exist in the column year and column death rate. Obviously we already saw this from in the previous examples that we have 476 values for each of these columns wh ich are the number of the rows that we have so the count would be 476 for either of them for year and death rate. The mean value here shows the average of the values i n the year column and in the death rate column that are listed here. Here you can se

e the standard deviation and then here you can see the minimum number of years, minimum value for the year which is 190 and the minimum death rate here is 11.4. T hen you have your first, second and third percentile. The second percentile of data of course that would be your median and you think about it as the first, second and thir d quartiles of your data. So what do they mean? 25% or first percentile of your data means 25% of your data, I mean the values in the year column, 25% of the values in t he year column will fall under this value or equal to it. Okay, so 25% of the values in t he year column will be equal or less than this value here. That's the first percentile. T hen you have your second percentile, 50% or the median, which means almost 50% of the data points or the values in the year column will fall under or equal to this val ue here, 1959. And last one, the third percentile, 35%, is 1989, which means again 75 % of the 75% of the values in the year column fall under or equal to this value. You c an interpret the values in the death rate column in a similar spirit. Of course the maxi mum value is clear, the maximum value in the year column 2018 so you know that th e minimum value in the year column 2018 so you know that the minimum value in th e year column is 190 and the maximum is 2018. Another thing that you can do is tha t you can use the transpose option here., will transpose the resulting table, that mea ns this table, into a horizontal view. So basically it transposes this matrix, this is a mat rix with these rows and these columns and these rows. The t option here transposes this matrix into this one. So it changes the position of the rows and columns. So the r ows become columns and the columns become rows. So the columns have become r ows, but the indexes of the columns have become the indexes of the rows, the type of the rows and the rows have become the colors. And yeah, so here you have a diffe rent way to display the same information description of the data frame, but this time horizontally instead of vertically. So it's just a display option.

41:12

Alright now that we've learned how to examine our data frame it's time to learn how to access the columns and rows of the data frame and we're gonna look into this. Ok ay let's see how we can use the dots notation to access the columns of a data frame. The data frame we're working with is mortality underline data which we refer to for si mplicity as mortality data. mortalityData.deathRate is going to access the death rate column of the mortality data frame. What does the head function do here? It's going to pick the first two values form of death rate from the data frame. So these are the t wo values that are displayed here. In other words, the head function is going to take

or access the first two rows and then from these two rows we are only interested in t he value of the death rate so it's going to only access the values of the death rate an d since this statement here is going to display these values we're getting these value s displayed here. Remember if we didn't access the death rate column, if we just said mortalityData.head2, it would display all the columns or the values of all the columns of the first two rows of the data frame. But since we have specified that we are only i nterested in the values of the death rate, we have only got the values of the death ra te. And of course the type function here if you call the type function for mortality dat a.deathRate it's going to return the type of the death rate which is a Pandas core seri es. There's another way we can access columns and that is using brackets. Instead of the dot notation we can use the brackets. So this way we can do that. We can say mo rtality data within the bracket we can specify the name of the column. It's deathRate. head. So we're accessing deathRate and we are getting the first two values of the de ath rate so the logic is the same only things changed here is the notation so the logi c is exactly the same as the dot notation. We've got the first two values. Similarly we can access two columns here so we're accessing year and death rate columns and ag ain we're getting the first two values with them so this time we get the values for the se two columns here so we're using the bracket notation here mortality data year an d death rate okay and again we can display the type of mortality data year and deat h rate what okay so this is this time we're actually getting in other words we're gettin g a sub data frame of the main data frame so our initial data frame original data fra me is mortality data but now we're actually getting, in other words, we're getting a s ub data frame of the main data frame. So our initial data frame original data frame is mortality data, but now we're only accessing the year and death rate columns of the mortality data. So if we try to get the type of mortality data, year, death rate, this is g oing to be a data frame, a data frame with two columns. What are the columns? Year and death rate. So unlike the dot notation, which would correspond to a series, the b racket notation corresponds to a data frame type. There's another way to access the columns using the lock accessor or LOC. And it's straightforward. You can use the na me of the data frame again, mortalitydata.lock. The first parameter here specifies the rows that we want to select and the second one specifies the columns. In this case, it' s a very simple example. We are selecting all the rows, so this column value here spe cifies that we are not filtering the rows, we just want to get all the rows from the dat a frame and these are the columns that we're selecting year and age group so we're only interested in these columns of the data frame but we want these columns from

all rows so this will give us all the rows but only year and age group columns alternat ively you could have filtered the rows as well so you could have said for example I w ant the mortality data with year greater than certain year and then you could have se lected whichever columns you would need to use.

04:37
Now let's see how we can access the rows. To access the rows we can use the query f unction, just one way. So a more tightly data is a data frame. Again dot query year e quals 1 900 is going to filter the data frame so that it only includes rows with year eq uals 900. Okay. Okay. We can also use the query function based on multiple columns using end So the way it works is that here's an example mortality data that query yea r equals 200 and age group Doesn't equal one to four years. So we're interested in a ny row from year 200 where the age group is not between one to four years. So thes e are the rows that we get. You can also access rows using query based on multiple c olumns with OR instead of AND. So there's just a different logic. So mortality data.qu ery year equals 900 or year equals 200. So it only includes, in other words, it only incl udes the rows whose year is either 900 or 200. And this is what you get. You can also access rows using the lock function or the lock accessor. Though at the end of the da y all these are implemented as functions in Python but we're not calling lock as a fun ction we're calling it as an accessor. So mortality data dot lock, zero, five, 10. What d oes this do? It's going to select rows with indices zero, five, and 10 from the mortalit y data frame. So we're picking specific rows and these rows are three specific rows wi th the indices 0, 5 and 10. This is what you get. And the next example shows that we are selecting a subset of the rows from the range specified here from index 4 to inde x 6. So 4, 5, 6 these are the rows that are selected from the data frame. And the last e xample is using again a range specifies the start of the range 0, the end of the range 20 and the step that is 5. So it picks index 0 followed by 5 then 10 and 15 and then 2 0 and these are the rows that are selected as a subset of the rows in the original mor tality data frame. So we can use the lock accessor to get access to a subset of the ro ws, either selecting them specifically by providing exact index or specifying a range o r specifying a range with a step. So you can skip some of the rules in the range. Here' s another example of using the lock accessor to access rows. So again, the name of t he data frame, totality data, the lock. And then here we specify a condition to filter th e rows so mortality data that year equals 1917 is going to give us all the rows from t he data frame with year equals this value 1917 this This is what you get. So we're get

ting this for different age groups. Now we can use a subset of rows and columns usi ng query function. We can also do the same thing using the log accessor. We'll see e xample of that as well. Let's first look at the query function. mortality underlined data .query year equals 190. This is going to select all the rows from the data frame with y ear equals 190 and from those rows it picks only the death rate column. And then it calls the head function which is going to display the first five values of death rates wi th year equals 190 and this is what we're getting here. And the reason we have only f our is because only those with the only four values available. Others would by defaul t get five values. That's what the head function is. And what else? The next example, mortality data query year equals 900, death rate. Okay, so this one, what it does is it does exactly the same thing, but with a different notation. Instead of using the dot n otation, but with a different notation. Instead of using the dot notation, it uses the br acket notation. So again, it selects all the rules from the data frame with year equals 190, and then it picks only the death rate column, this time using a bracket notation i nstead of a dot notation. This notation gives you more flexibility because you can sel ect a subset of the columns instead of one specific column. All right, accessing a sub set of rows and columns using query, we continue on that. mortalityData.query equal s 190, the death rate. So with this notation, we are accessing again the death rate col umn of all rows with year 190 but you can see here we are embedding death rate wit hin two brackets. So this bracket is open here and closed here. There's another one o pen and closed here. So there's a bracket that is open here and closed here. There's another one that's open and closed here. So there's a nested bracket. You don't nee d the nested bracket if there's only one column you want to access. But if you want multiple columns, if you want to access more than one column, then you need the se cond or the nested bracket. So that's why this notation gives you more flexibility. You can have multiple columns listed here. In this example you are accessing the age gro up and the death rate columns of the data frame for the rows with year equals 190. S o now it is the query function filters, the data frame base to only include the rows wh ose year is 190 and then accesses the age group and the death rate of those rows. A nd this is achieved through the bracket notation using these nested brackets.

53:52

As mentioned earlier, we can also access a subset of rows and columns using the loc k accessor. So instead of query, we can also use the lock accessor. Mortality data.lock , zero, five, 10, and then age group and death rate. So this gives you a horizontal and

vertical slice of the data frame. It gives you these specific rows, zero, five and 10, thes e are the indices of the rows. It gives you the row with the index zero, the row with t he index five and the row with the index 10 and from those rows it extracts the age g roup and the death rate of each row and then displays them so I get this age group and death rate for these three rows displayed here. Let's look at the next example. T he next example, again, accesses rows from index four to six, so four, five, six, saying an accesses the age group of an accesses the, oh, okay. So this one also uses a range to access the columns. So instead of listing the columns, it is accessing the columns using a range. So it says I want to access any column between age group and death rate. So anything that is between them. Well in this case there's nothing between the m they come in the sequence that's why it's showing only the t2. But if there was a c olumn between them it would also be included because this is a range. It says I want any column from edge group to death rate, anything that comes between them wou ld have also been included. Similarly specifies the rows by range from index 4 to 6 w hich are listed here. from index four to six, which are listed here. All right, so there's a nother accessor that we can use, this ilock, to select a subset of the rows and the col umns in a data frame. There's a difference between ilock and lock, and it is in the loc k accessor, we would use indices of the rows and the labels of the columns to select a subset of the rows and the labels of the columns to select a sub-server of rows and columns. In the case of the ILAC accessor we use indices for both rows and columns. So let's look at the first example to get what I mean. In the first example here we're s pecifying this specific rows 4, 5, 6, these are the indices of the rows that we are select ing from the data frame, mortality data, and one and two are the indices of the colu mns that we're selecting from this data frame. So we're selecting rows four, five, six, and from these, and from the columns, we're selecting only column one and two. Col umn one and two correspond to age group and death rate. So this is the resulting d ata frame. So instead of providing the label of the or the labels of the columns that we want to select, we're providing the indices of the columns. Okay. There's another example here that we are using a range for both rows. We're specifying two ranges fr om the rows and the columns. We're selecting every row between four and six. I kno w, I know this seven here is not going to be included and don't ask me why. Yes, irrat ional things can happen in designing functions. I know it doesn't match with what w as in the lock function. In the lock function, if you had four to six, index six would als o be included. So you would have four, five, six. But here, like if you had four to seve n, four, five, six, and seven would be included. But in the case of iLog, only four, five

and six are included. So seven is like the end of the range, but it's not included. As y ou can see only four and five and six are included. Similarly, the second part, one, tw o, three, only one and two are included. That means the first and second column. Th e last column is not included. Again, this is just a design choice. It has no logic behin d it, as far as I'm concerned. Maybe this is the result of thinking rationaly, acting ratio naly. I don't know. But this is what it is the result of thinking rationaly, acting rational y, I don't know. But this is what it is, this is how it works and to work with any axis or function you just need to know how it works and sometimes they're not necessarily f ollowing logic. So here in this case when we say 4 to 7 that doesn't include 7, so it's 4 5 6. When you say between 1 to 7 that doesn't include 7 so it's 4 5 6 when you say between wolf 1 to 3 that doesn't include 3 so it's 1 and 2 just have that in mind if yo u're going to use a lock accessor.

58:55
ok here's a question for you what's the difference between lock and iLock This is not a difficult question, probably not even an important question, but rather this is to m otivate you to start using ChatGPT to ask questions, like questions as simple as this o ne or as sophisticated as it could get. So go to ChatGPT, ask this question and ask Ch atGPT to explain it to you with some examples. And if your ChatGPT is as lazy as min e, you might get answer like this, but if you write a better prompt or direct ChatGPT t o give you a better answer, a more comprehensive answer with examples, you might get something more interesting. So I'm going to pause here and emphasize that. Wit h the new technologies around, with generative AI, not specifically ChatGPT, but all t he generative AI tools being around that are able or capable of understanding text a nd reasoning, there's less and less need for understanding syntax. And there's less a nd less need for memorizing what different programming languages do or how diffe rent things are written. Of course, you need to familiarize yourself with these things s o that you can use them, you can modify code, you can understand code. But to me morize them, in my opinion, is a waste of time. So you need to learn. you need to lea rn how to work with these technologies together and effectively and efficiently deliv er the solution that you need to deliver. So this highlights the importance of using g enerative AI throughout this unit. I suggest starting with asking questions. In other w ords, asking a good question is a big step forward to learning. To interact with ChatG PT will teach you how to ask good questions. Sometimes you ask a question and you realize that you should not ask the question the way you asked and it helps you to i

mprove your understanding of the problem. So asking a good question is a big part of learning and interacting with generative AI tools such as ChatGPT will help you to achieve that which means it will greatly contribute to your understanding of this unit. So please consider this seriously and try to use it throughout your journey in this uni t.

01:02:08
Okay, we're going touch a little bit on how to prepare the data. We're going to come back to data preparation later on, but here we focus on some of the essential things that you need to know for preparing your data. And we know that in the five phases of data analysis, data preparation comes after data cleaning. But here we just wanna give you an overview of the things you can do to prepare your data. All right, so one of the things that we can do with our data frame is to sort our data based on differe nt parameters. Let's look at some examples. Data frame name, that sort values death rate so this is going to sort the rows in mortality data based on death rate in descen ding order because it's setting the value of ascending to false that means the sort is going to be done in descending order. And then we're taking the first three rows of t he sorted data frame. So you can see that this is what we get. In other words, anothe r way to describe the result is we are extracting the rows from mortality data with the largest death rates because we are sorting the data in descending order so if we star t with the largest death rate and we go you know we go down and we are picking th e first three so we're picking their rows with the largest death rates the three rows wi th the largest death rates. The three rows with the largest death rates. So top three d eath rates. And then the next example is extracting, is again is sorting the data, that means the rows in the data frame based on year and then based on death rate. So in itially the sorts would be data frame based on year and then based on death rate so i nitially the sorts would be carried out based on year and then based on death rate s o for so within each year the entries the rows will be sorted based on death rate so if you take for example the first three rows they correspond to the year 190 and becau se that's the smallest, that's the earliest year that we have in our data that is 190 and it goes all the way up to 2018 why the sort function is returning the, why the head fu nction is returning the smallest year is because we are doing the sorting base in asce nding order that's by default sort values by year and death rate is going to sort the d ata initially based on year in ascending order and then based on death rate in desce nding order within each year okay so both of these are sorted so both of these para

meters are used to sort in ascending order by default. So for a given year 190 obviou sly the rows are sorted based on death rate in ascending order. So we're going from 298.3 to 484.8 increasing and sorted in ascending order. And we're picking the first t hree rows, so we're getting the rows that correspond to the smallest here, 190, and y ou can see that the rows are sorted based on death rate in ascending order. What if you want to change this? What if you want to sort the death rate in descending orde r? We could do that by passing the parameters to the values to the parameter ascen ding. So we can do it as follows. Mortality data, the sort values, same as before, base d on year, then based on death rate. So the difference here is that the year will be so rted in ascending order, but the death rate, sorry, the data will be sorted based on ye ar in ascending order and based on death rate in descending order. Because the valu e of ascending is set to false for death rate. So again, the rows will be sorted based o n year initially in ascending order and then based on death rate in descending order. So for a given year the rows for that particular year are sorted based on death rate in descending order. Okay let's look at the results here you get five rows this time the h ead function is returning five rows by default so you're getting five rows. They corres pond to year function is returning five rows by default, so you're getting five rows. T hey correspond to year 190, and then there's one row that corresponds to year 1901. And then you can see the rows are within this year, within 190, the rows are sorted b ased on death rate in descending order. In descending order. This one of course is a new year. So within this year, that year means 1901. So within this year also, their ro ws will be sorted based on, in descending order. If you print more of this year, you w ill see that the rows are also sorted in descending order.

01:07:51
Okay. How to apply statistical methods? Well, we can apply statistical methods to ea ch column by using the dot notation for example so mortality data dot death rate th at mean is going to return the average or the mean of death rates in the mortality da ta frame. We could apply that to any other numerical columns and you could also ap ply a statistical method to a group of columns in this case for example we are applyi ng the max function to age group and death rate which is going to return the maxim um of age groups and death rates for each of them. We can also use the count functi on to count how many values exist for each column. In this case we have 476 values so it's not counting the unique values it's just counting all the values including the re dundant values or repeated values in each column. And you can see that you have 4

76 values in each column corresponding to the number of the rows, we have 476 ro ws which means we don't have any null values, we have values for each columns. Ok ay to continue applying statistical methods, we have another example here mortality data quantile 0.1 to 0.9 is going to calculate the 10th and 90th percentile for each nu meric column in the mortality data frame. So you can specify the quantile the quantil e 0.1 which corresponds to the 10th percentile and 0.9 corresponds to the 90th perc entile and we know that 90th percentile means the point at which 90% of the values are less or equal that value at that particular point. In this case, for example, for deat h rate, we can say 90% of the death rates are less or equal 430.85. rates are less or e qual 430.85. Another function we can use is cum sum. We are applying this cumulati ve summation of any numerical column in this case death rate. We're applying the cu mulative summation function to the death rate which is going to return the cumulati ve sum of the death rate column which means it starts from the first death rate then the second one the second value would be summation of the previous death rate wit h the new death rate and so on so forth. So you calculate the cumulative sum. So thi s is how we can use the quantile method. Import pandas as pd of course df equals p d.dataframe. It's creating a data frame for us with two columns and b with their corre sponding values. And if you're interested in calculating the first, second and third qu antiles of the data that means a point percentile, 25%, 50% and 75%. And 50% perce ntile or 0.5 quantile is also the median. So you can see the values here. The quantiles are calculated for 0.25, 0.50, 0.75 for both columns A and B. For column A the 25 per centile or quantile 0.25 is 2 and for column B is 20. You can see these are the values i n column B and the values in column A. Okay you can also apply some column arith metics to the columns and then create new values from the columns for maybe mor e for other calculations or for other type of analysis that you might need later. Here we have two examples. The first example creates a new column in your data frame n amed mean centered which did not exist before and the values in these columns are calculated for each row by deducting the mean of the death rates, the mean of all de ath rates in the death rate column from the death rate in that particular row. So this i s how we can calculate the mean centered value for each row and then we get this w ay we can get a new column for name mean centered containing the values that are calculated as specified here. The second example does not create a new column inst ead it replaces the values in an existing column that is death rate, so this column is al ready here, we are replacing the values by dividing them by whatever numbers, 100, 000, so that we can convert it to the rate per 100,000 population, for instance. So the

values are replaced by the new value that's calculated here by dividing the death rat e by this number. So two different types of column arithmetic that we saw here.

01:14:00
We can also modify the string data in a column. How can we do that? Let's look at th e first example. Mortality data age group that replace. So this is going to replace cert ain values in this case one to four years and five to nine years to another format to 0 1 dash 0 4 years. So instead of using the number in the initial format that is for exam ple 1, we use 0 1. So 4 is 0 4. This is just formatting purposes. So we're replacing ever y value in a column in the age group column, that is 1-4 space years. We replace that value with 01-04 space years. And similarly we replace the other value for five to nin e years. And the parameters in place is set to true which means that this is going to i mpact the original data frame. That means it's going to impact the mortality data fra me. If this parameter was not set to true, if it was set to false, we would have got a n ew data frame. It was a copy of the original data frame with the updated values for t he age group column and this would have not affected the original data frame. So w e might choose one or another one, like one or two to set in place, the true or false, depending on what we want to achieve. Whether we need the original data frame or not. Another example here replaces values in the age group column using the dictio nary. What it does is it just maps 1 to 4 years to 0, 1 to 0, 4 years and maps values 5 t o 9 years to 0, 5, 0 to 9 years. So instead of to replace and value, which are two differ ent lists, so mapping this way we're just using a dictionary to do the mapping to mo dify the string data in the columns. Alright. The next topic, how to shape the data. So metimes we need to change the way we represent the data for different reasons. So metimes it makes it easier to visualize the data or analyze the data. So we might nee d to reshape the data or analyze the data. So we might need to reshape the data in our data frame. One thing that we might need to do is to change the index. As discu ssed earlier the indices are sequential numbers by default. So each row is uniquely id entified by a number. But we can change that. So we can set index. We can set the in dex to be a specific column, in this case year. So this functions that index is in this ex ample is changing the index to be the year instead of sequential numbers and you c an see that when we display the first two rows of the data set year is highlighted her e as the index. Here we have an example of ating the index. When we reset the index we change back index to the sequential numbers that are used by default and inplac e equals true means it's going to impact the original data frame in this case mortality

data this another example mortality data equals mortality data set index year age gr oup and verify integrity true so what this example does is it uses a combination of co lumns in this case year and age group together to uniquely identify the rules. So yea r and age group together will be used as the index and we're setting the verify integr ity true so which means the integrity of the data frame will be checked, which means if we have... Well, as mentioned, we're using a combination of columns, in this case, y ear and age group, as the unique identifier of each rule, right? But if we have a reput ation of a specific combination of year and age group in the data or in the data fram e, it means we have one index for two or more than two rows. That will hamper the i ntegrity of the data frame because that would change the definition of index. Index i s supposed to uniquely identify a row but if you have identical indices for multiple ro ws, it's no longer functioning as an index. So by setting the refer integrity to we asser t that we want that to be checked to make sure that the integrity of the data frame is maintained. Otherwise we need to change our index to something that makes sense, something that can uniquely identify the rows. Again we can reset the index which w ill change the index back to the sequential numbers.

01:20:07
Pivoting data, so pivoting and melting are two operations that we can perform on th e data frames to change the way that data is presented. Pivoting is to transform rows into columns or Y-Systems in a dataset and that allows to restructure data to make it more useful for analysis or reporting purposes. To pivot data, you typically identify a column of interest that you want to become the new column headers and a second c olumn that contains the values that should be placed under each new header. So her e's an example. Mortality-wide is a data frame, is a wide representation of the datafr ame. So by pivoting we're going to create a wide representation of the data frame. T his is called so-called wide representation. So mortality-wide is a dataframe that will contain a wide representation of mortality data. So mortality data that pivots is goin g to create a wide representation of mortality data. mortalityData.pwt is going to cre ate that wide representation. mortalityData.pwt index equals year. index equals year means year is going to be used as the index in the new representation. The column o f interest would be age group. So the values of the age group are going to be used a s the columns in the new data frame, in the white data frame. So we're taking age gr oup, we're taking the values of age group, and those values would be the new colum ns. You can see here 0 to 4, 5 to 9, 10 to 14, 15 to 19. These are the values of each gr

oup in the original data frame. The values corresponding to the age group column in the original data frame. But in the new data frame they will become the headers of t he data frame or the columns of the, the headers of the columns of the new data fra me or the columns of the new data frame or the labels of the columns of the new da ta frame or the columns of the new data frame or the labels of the columns of the ne w data frame, however you want to call it. And the values that will be displayed in th e new data frame, that means in the white data frame are the death rates. So here yo u can see that. So the index is here, year is used as the index. Columns, the columns of the wide data frame are the values of the age group in the original data frame an d the values are the death rates, as you can see here. So it's a new representation of the original data frame with these settings. This is called pivoting. So we are doing th e pivoting based on this column. This is the column of interest specifying the index a nd the values that are going to be included. The next example is essentially the same example which is just breaking the line into multiple lines, breaking the statement int o multiple lines. This is just an example of that. Otherwise everything else is the same . Melting is a process of transforming a dataset from a wide format to a long format. So in the pivoting went from a long format to a wide format. Now we're doing the re verse. We're going from the wide format to the long format using melting. That typic ally involves taking columns of data and converting them into rows. It can be useful f or data analysis and visualization. Here's an example of melting. This is our original d ata frame, not original data frame, it's the white data frame that was created from th e original data frame through pivoting. Now we use the melt function of the data fra me. So we apply the melt function to a mortality-wide data frame. melt function to a mortality wide data frame we set the ID varus parameter to year we set the value var us to a subset of the column so we're only selecting these two columns from the fro m the wide data frame we're not picking all the columns we could have picked all of them, it's just a choice here. We're interested only in these two columns. This column and this column. And we're assigning this as the value varus. So these are going to b e the values for the age group. And so the var name is age group and the value nam e is death rate so the death rate is going to contain the death rates the variable nam e is age group which is going to be one of these two values and so this is the values of age group and ID where it's set to year. So we have year, age group and death rat e here. Year, age group and death rate. And the values of age group are specified by value whereas list here. So there's a list that specifies all the values of value errors, w hich could be repeated based on for different years. So in other words, each year mi

ght have a value error of one to four or five to nine. And of course, they're correspon ding death rates. So here we're doing the melting, we're changing the wide format t o a long format. We're transferring a wide format data frame to a long format data fr ame. And yeah here we are displaying maximum of four rows, that means two rows f rom the beginning and two rows from the end from the long data frame. You can se e it here. We're going to see more examples of these later when we talk about data preparation, but for now this is enough to give you an idea of how you can change d ata from one representation to another representation for different types of analysis.

01:27:10
Speaking of analysis it's now time to talk about analyzing the data. As part of your d ata analysis you might be interested to group your data so that you can apply differe nt aggregation functions to the data or to the groups. So you might be interested, fo r example, to group your data, your mortality data based on age group. In this case, you have four different age groups. You have four different age groups. So if you gro up your database on age group, you will have your roles categorized or summarized under these age groups. And then you can apply a function like mean. This is an exa mple of an aggregate function. So we're applying the mean function, which is going to calculate the average of the of the values that correspond to different age groups. And this is going to apply the mean function to all numerical columns. In this case, m ean centered and death rates are numerical, also the year. So this is going to calculat e the average year death rate and mean centered corresponding to each age group so two things are happening here first grouping your records or your rows based on age group and then applying the mean function to them so the average will be calcu lated per group. The average of which columns? The numerical columns. We haven't specified any columns here so it's going to automatically only consider numerical col umns that is year, the thread, mean center. The next example calculates the median of the numerical columns for each year. So we're doing the grouping based on year i n this case and since we are doing the grouping based on year the remaining numeri cal columns are death rate and mean centered so the median is computed for death rate and mean centered and the listed here for different years. So again two operatio ns first grouping and then applying the median function to calculate or compute the median of the numerical columns that mean death rate and mean center for each ye ar. Here we have another example, mortality data.group, year, age group. So, okay, t his is an interesting example. We are doing the grouping based on two columns this

time. So grouping will be performed initially based on the year column and then bas ed on age group. So we have two parameters to do the grouping based on and then we applying the count function to calculate how many rows fall under each fall unde r each group and each group is uniquely identified by a combination of year and age group. So we have a group here that corresponds to year 190, age group of 1 to 4 y ear old and you have another group that corresponds to year again 190 and age gro up of five to nine years and so on so forth and you can see that we have for example one item that falls here falls under this group and that one item that falls here so the count here is calculated for, again, the numerical columns and the numerical column s in this case are death rates and mean centered. So for both of these columns, all w e're calculating is just how many rows are falling under each of these. So number on e here tells us there's only one row that corresponds to this group and is again num ber one here tells us that there's only one row that corresponds to this group that m eans to this here and this particular edge group and so on so forth. Alright so here w e can see an example of applying aggregation functions explicitly using egg or age f unction and in this case we're applying two specific functions mean and median to t he numerical columns corresponding to different age groups. So initially we're formi ng the groups around age groups and then we're applying initially we're forming the groups around age groups and then we're applying, we're calculating the mean and median for each of the numerical columns that correspond to these age groups. Tha t means year, death rate, and mean centered. So for each of these, year, death rate, a nd mean centered, we're calculating the mean and median as listed here. Here we ha ve another interesting example. We are performing the grouping based on age grou ps again, and then we are selecting a specific column this time that is death rate. So t his time we're not applying the aggregation to our aggregate functions to all the nu merical columns. We're applying them to the specific column, which in this case is de ath rate. So perform the grouping, select the column that you want to work with, an d then apply the functions that you want to apply to this column, mean, median, sta ndard deviation, and N-unique. So we calculate mean, median, standard deviation, a nd unique for specific column that is selected here, that is death rate. We could have selected a subset of the columns by including them in a list here using a nested brac ket. But in this case, we're only interested in one column and the column is selected here and we're calculating these statistics for this particular column corresponding to different age groups. Another example mortality data that group by year so this time we are grouping the rows based on year and for each year, we are calculating the ag

gregation functions listed here for specific column that is death rate. So we are doin g the grouping based on year and then we are applying these functions all listed her e for the death rate as listed here. So similar to the previous example.

01:34:05
So, one of the ways that we can easily visualize the data is using a line plot. So morta lity data.pivot here is going to create a wide format of the data frame mortality data. The column of interest is age group so it's going to create a data frame in which the columns are going to be the values of age groups and the index is going to be the y ear and we're extracting the death rate from that data frame and plotting the death r ate. So we're plotting the death rate against the index in this case and the index is ye ar. So we can see that the death rate is plotted against the against the year for differ ent age groups again the columns are representing different values for the age grou ps therefore we're gonna get one, two, three, four lines in the plot, each correspondi ng to an edge group, which is one of the columns of the wide data frame. And here's an example of using a bar chart to visualize a data frame that is created from the ori ginal data frame of mortality data. The data frame that we are visualizing is a data fra me that groups all the rows in the mortality data based on age group and then calcu lates the mean, median, and standard deviation for the death rate. So the resulting d ata frame is visualized using plot.barH specifies that this is going to be a horizontal b ar chart and as you can see you have a bar chart where a mean, median and standar d deviation of the death rates are visualized against different age groups. Okay, here' s a list of some of the references we have used for preparing this lecture and... End of the lecture.